

DATA 622 Lab 4: NBA Player Evaluation Using Ridge Regression and the Lasso

Joao De Oliveira

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pickle import TRUE

from sklearn.linear_model import LassoCV
from sklearn.utils import resample
from sklearn.linear_model import RidgeCV
from sklearn.model_selection import cross_val_score
from sklearn.metrics import mean_squared_error
```

Overview

Complete this assignment by answering the following questions using code, text descriptions, and mathematics in a quarto markdown document. Render your .qmd to a pdf and submit both the qmd and pdf files on brightspace.

In this lab you are going to be using resampling techniques (cross-validation and the bootstrap) along with regularization (ridge regression and the lasso) to rank basketball players based on their performance during the 2022 NBA season. To get the most out of this lab, you do not need to be an expert on the NBA, but you do need to pay attention to the following background information.

Basketball is a sport played between two teams. Each team has 12 total players, but only 5 are on the court for each team at a given time. The goal of the game is to score more points than the other team during the 48 minutes of game time. Points are scored by “shooting” a ball into a 10 foot tall metal hoop called the “basket”. If you want to watch the gameplay, [skim this video](#). Or, you can watch [this 8 minute video that explains the rules](#). Traditionally, basketball players have been evaluated based on their individual statistics, such as the number of points that they score in a game, the number of “rebounds” they get (a rebound is when a

player catches the ball after a missed shot), the number of “steals” (when a player takes the ball from the other team), and more. However, basketball is a complicated team sport, and the actions of all five players on a team contribute to these individual statistics.

The goal of capturing these more abstract contributions to success has led to the concept of “plus-minus”, which looks at how the presence or absence of a player on the court correlates with the scoring margin of the team. The idea is to model what happens in a basketball game by assigning each player on both teams a “coefficient”, which will be called ‘RAPM’ during this homework assignment. ‘RAPM’ stands for Regularized Adjusted Plus Minus, and it measures the contribution of each player to the expected point differential between the two teams. A player with a RAPM of 3 is expected to add 3 points to their team’s net performance for every 100 possessions of a basketball game (a possession is defined as a period of the game where a single team has control of the ball, games consist of a series of alternating possessions and there are usually about 200 possessions in a game, 100 per team).

Suppose that two teams are playing against each other, and that the ‘lineups’ of the two teams stay the same for a certain number of possessions, call it ‘num_pos’. This period of time with constant lineups is called a ‘stint’, and we can model the probability distribution for the point ‘margin’ (defined as $(\text{points_home} - \text{points_road}) \cdot \frac{100}{\text{n_pos}}$) in the ‘stint’ using the following formula:

$$\text{margin}_i \sim \text{Normal} \left(\sum_j c_{ij} (\text{RAPM})_j, \sigma^2 \right)$$

Here, the sum is over all the different players in the dataset, and the coefficient c_{ij} is +1 if that player j was on the court during stint i and playing for the ‘home’ team, and -1 if they were on the court for the ‘road’ team. The coefficient is 0 otherwise. The ‘home’ team and ‘road’ team distinction allows us to use the same data for players who were playing against each-other. The ‘margin’ variable is the point differential during the stint normalized to points per 100 possessions.

The dataset to fit this model is contained in [nba_stint_data.csv](#). Let’s take a look at it:

```
nba_stints = pd.read_csv("https://raw.githubusercontent.com/georgehagstrom/DATA622Spring2026/main/nba_stint_data.csv")

nba_stints.head(10)

# target
y = nba_stints["margin"]

# features
X = nba_stints.iloc[:, 7:]
```

Here you can see that each row corresponds to a stint where the players on the court didn't change. The 'n_pos' variable is the number of possessions that took place during that stint. The 'home_points' and 'away_points' are the number of points scored by the home and away teams respectively. The 'minutes' describes how long in minutes the stint lasted, and the 'margin' is the target variable (defined earlier). The features in column 7 onwards are labeled by player ids, and the value in each cell corresponds to the c_i values for that stint (whether the player was on the court for the home team, road team, or neither). You can recover the identity of the players from the 'player_id' file.

In the following lab you will be exploring different ways of calculating the 'RAPM' model coefficients and interpreting them in terms of player skill.

Problem 1: Ridge Regression for Inference

- (a) **Ordinary Linear Regression:** Use ordinary linear regression to fit the model described in the overview. Use cross-validation to estimate the out of sample root mean squared error and compare it to the in sample error, you may use 'RidgeCV' with $\alpha = 1e-8$ to keep consistency with the rest of the assignment, or use 'LinearRegression'. Make sure 'fit_intercept=True', the intercept corresponds to the home court advantage, and do not use 'sample_weight'. Does the difference between in-sample and cross-validated mean squared error suggest a major problem with overfitting?

Fit Linear Model - OLS via Ridge

```
# fit linear model
model = RidgeCV(alphas=[1e-8], fit_intercept=True)

model.fit(X, y)
```

```
RidgeCV(alphas=array([1.e-08]))
```

In-Sample Root Mean Square Error

```
y_pred = model.predict(X)

rmse_in_sample = np.sqrt(mean_squared_error(y, y_pred))

print("In-Sample RMSE:", rmse_in_sample)
```

In-Sample RMSE: 69.62888762317858

Cross-Validation RMSE

```
cv_scores = cross_val_score(  
    model,  
    X,  
    y,  
    scoring="neg_mean_squared_error",  
    cv=5  
)  
  
rmse_cv = np.sqrt(-cv_scores.mean())  
  
print("Cross-Validated RMSE:", rmse_cv)
```

Cross-Validated RMSE: 71.64080821359738

The in-sample RMSE is around 318, which is much lower than the cross-validated RMSE of 9779, indicating a big difference between training and test performance. This means that the model is definitely overfitting the data. While it fits the training data pretty well, it does not do the same with unseen data. This is probably because of the large number of player features and multicollinearity.

- (b) **Examining ‘RAPM’ Coefficients:** Create a dataframe with the player-ids, the RAPM coefficients, and join it with the player names (from the data file shared earlier). Use the stint matrix to calculate the number of minutes that each player played (‘minutes’ variable) and add that to the data frame too. Sort the players in descending order by ‘RAPM’ and print the top 20 players by ‘RAPM’. What do you notice about their minutes played? Look up the names of a few of the top players on the internet- are they regarded as top NBA players? Make a scatter plot of ‘RAPM’ versus minutes-played.

Extract RAPM coefficients and Load Players Names

```
# extract coefficients  
rapm = pd.DataFrame(  
    "player_id": X.columns,  
    "RAPM": model.coef_  
)  
  
# load players names  
players = pd.read_csv(  
    "https://raw.githubusercontent.com/georgehagstrom/DATA622Spring2026/main/website/assignm"
```

```
)
```

```
players.head()
```

	player_id	player_name
0	1630173	Precious Achiuwa
1	203500	Steven Adams
2	1628389	Bam Adebayo
3	1630534	Ochai Agbaji
4	1630583	Santi Aldama

Merge RAPM and names

```
players["player_id"] = players["player_id"].astype(str)
```

```
rapm = rapm.merge(players, on="player_id", how="left")
```

```
# minutes played
```

```
minutes = []
```

```
for player in X.columns:
```

```
    mins = (nba_stints["minutes"] * (nba_stints[player] != 0)).sum()
```

```
    minutes.append(mins)
```

```
rapm["minutes"] = minutes
```

Top 20 players

```
top20 = rapm.sort_values(by="RAPM", ascending=False).head(20)
```

```
top20[["player_id", "player_name", "RAPM", "minutes"]]
```

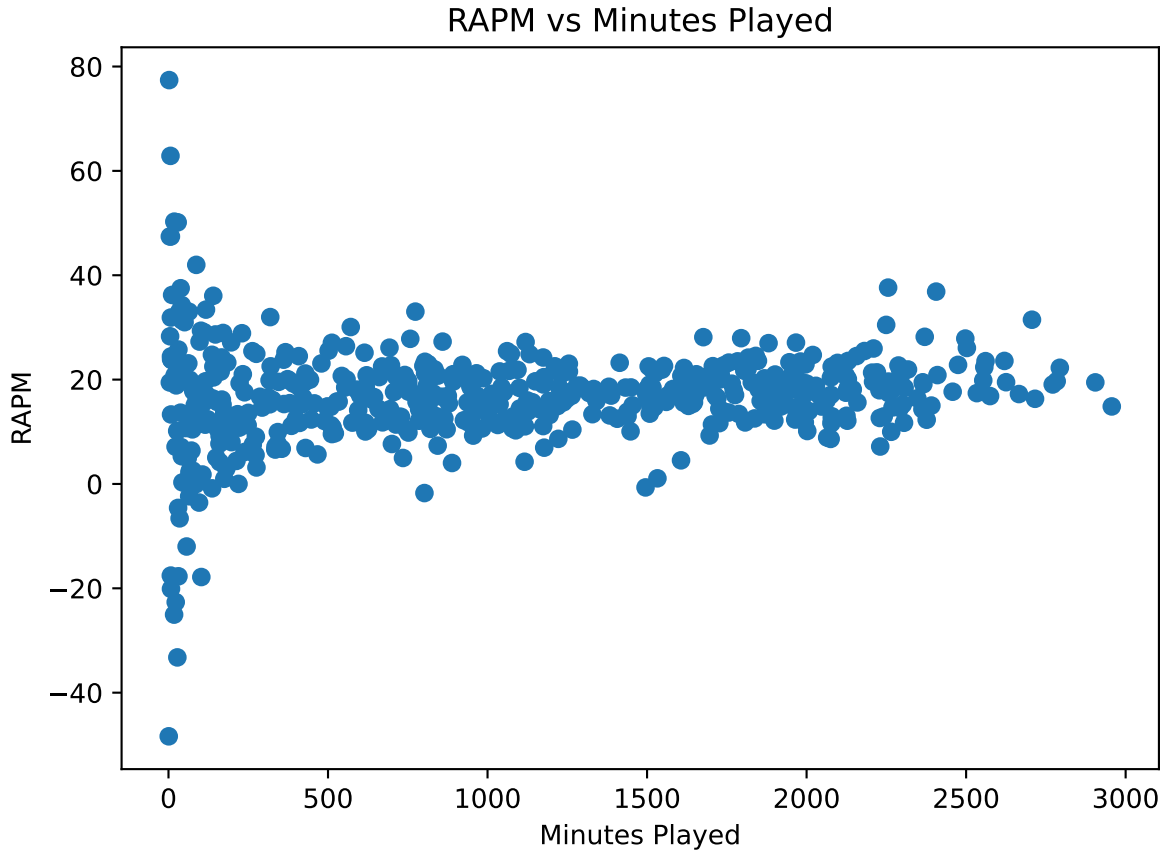
	player_id	player_name	RAPM	minutes
533	1630649	Stanley Umude	77.399013	2.12
382	1630648	Jordan Schakel	62.877108	6.24
279	1630250	Marko Simonovic	50.272182	18.32
458	1628993	Alize Johnson	50.138384	28.80
531	1629126	Deonte Burton	47.434768	7.55
535	1631495	Donovan Williams	47.427046	4.25
442	1630600	Isaiah Mobley	41.990297	87.07
258	203999	Nikola Jokic	37.629364	2255.54
447	1631112	Kendall Brown	37.508835	38.13
218	1628973	Jalen Brunson	36.864243	2406.14
529	1631220	Dereon Seabron	36.223242	11.37
520	1629677	Luka Samanic	36.063442	140.33
455	1629662	Mfiondu Kabengele	34.273259	41.68
404	1631199	Ron Harper Jr.	33.437517	46.14
510	1641645	Xavier Cooks	33.412151	117.48
456	1629653	Devon Dotson	33.312653	50.52
226	1630540	Miles McBride	33.029755	773.80
325	1631213	Tyrese Martin	33.015002	63.91
511	1629714	Jarrell Brantley	32.954130	33.13
536	1630644	Mac McClung	32.606134	44.14

Scatter Plot

```
plt.figure(figsize=(7,5))
plt.scatter(rapm["minutes"], rapm["RAPM"])

plt.xlabel("Minutes Played")
plt.ylabel("RAPM")
plt.title("RAPM vs Minutes Played")

plt.show()
```



The top 20 RAPM players are mostly players with a small number of minutes played. For example, several of the highest-ranked players (such as Stanley Umude, Jordan Schakel, and Deonte Burton) played only a few minutes during the season but have very large RAPM values. This suggests that the model is producing unstable and inflated coefficients for players with limited data.

While some top NBA players like Nikola Jokic and Jalen Brunson also appear in the top RAPM rankings, most are not top NBA players. This shows that linear regression is overfitting and assigning unrealistic importance to players with small sample sizes.

The scatter plot shows exactly the same pattern, showing that players with fewer minutes tend to have more extreme RAPM values, while players with higher minutes have more moderate and stable estimates. So, the coefficients must be stabilized.

- (c) **Ridge Regression RAPM:** The results of (b) suggest that the model is attaching extreme values of 'RAPM' to low minute players, something which can be potentially fixed with regularization. Define a vector of regularization parameters 'alpha' on a logarithmic scale between 10^{-2} and 10^5 (look up 'np.logspace'). Make this vector contain at least 10 but not more than 200 values of 'alpha' (pick based on how fast your computer

is). Use 'RidgeCV' to fit a ridge regression model, selecting the model with the best value of the hyperparameters. What value of 'alpha' is optimal? Next repeat the same calculation as you did in part (b) (you could create a function or just copy the dataframe and replace the old 'RAPM' with new 'RAPM'). Look up some of the top players that your model identified, are they well regarded by the NBA? Make a scatterplot of 'RAPM' versus minutes played.

Alpha Grid

```
alphas = np.logspace(-2, 5, 40)
```

Fit RidgeCV

```
ridge_cv = RidgeCV(alphas=alphas, fit_intercept=True, cv=5)

ridge_cv.fit(X, y)

print("Optimal alpha:", ridge_cv.alpha_)
```

Optimal alpha: 701.7038286703822

New RAPM

```
rapm_ridge = pd.DataFrame({
    "player_id": X.columns,
    "RAPM": ridge_cv.coef_
})

rapm_ridge = rapm_ridge.merge(players, on="player_id", how="left")

rapm_ridge["minutes"] = rapm["minutes"]
```

Top 20 Players

```
top20_ridge = rapm_ridge.sort_values(by="RAPM", ascending=False).head(20)

top20_ridge[["player_id", "player_name", "RAPM", "minutes"]]
```

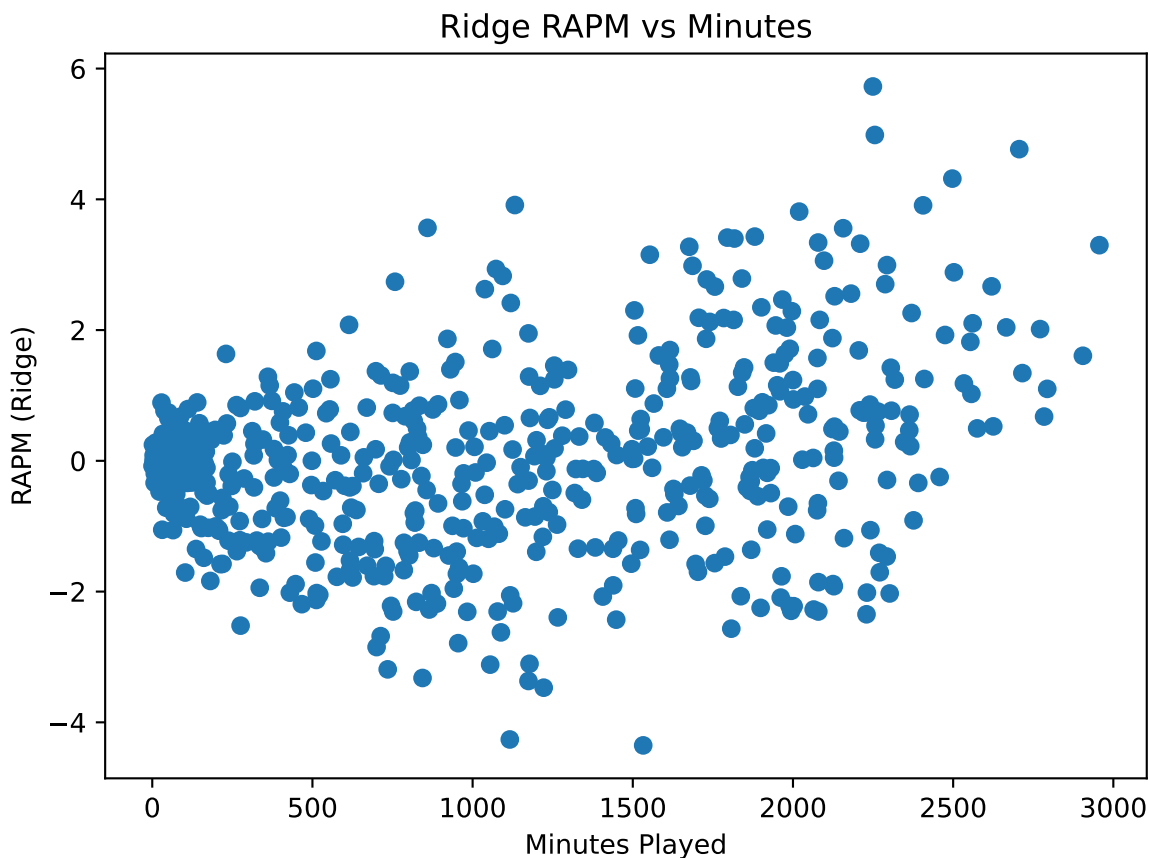

	player_id	player_name	RAPM	minutes
156	203954	Joel Embiid	5.726135	2249.47
258	203999	Nikola Jokic	4.984335	2255.54
48	1629027	Trae Young	4.767390	2706.40
99	1627783	Pascal Siakam	4.317015	2497.48
318	201567	Kevin Love	3.912561	1131.82
218	1628973	Jalen Brunson	3.908070	2406.14
2	203110	Draymond Green	3.812067	2019.37
419	1629627	Zion Williamson	3.563620	859.13
242	201572	Brook Lopez	3.557829	2156.65
167	203076	Anthony Davis	3.429990	1880.81
272	1629632	Coby White	3.414445	1795.01
306	202695	Kawhi Leonard	3.400299	1817.22
16	1628401	Derrick White	3.339307	2078.65
406	1629636	Darius Garland	3.321377	2209.82
217	203944	Julius Randle	3.298562	2956.55
417	1626167	Myles Turner	3.274717	1676.32
339	1630198	Isaiah Joe	3.152674	1553.19
243	201950	Jrue Holiday	3.061368	2097.00
235	1630532	Franz Wagner	2.993732	2293.49
260	1629008	Michael Porter Jr.	2.983194	1686.43

Scatter Plot

```
plt.figure(figsize=(7,5))
plt.scatter(rapm_ridge["minutes"], rapm_ridge["RAPM"])

plt.xlabel("Minutes Played")
plt.ylabel("RAPM (Ridge)")
plt.title("Ridge RAPM vs Minutes")

plt.show()
```



The optimal value of the regularization was around 701.7, showing that a relatively strong amount of shrinkage was needed to stabilize the model. Compared to the previous results (ordinary linear regression results), the ridge RAPM values are much more reasonable/close to what it expected and now it isn't mostly dominated by low-minute players. The top-ranked players now are well-known NBA players like Joel Embiid, Nikola Jokic, Trae Young, Pascal Siakam, and Jalen Brunson, all recognized as top players. So, the ridge regression produces more realistic and reliable player rankings.

It is also noticeable that the top players now have substantial minutes played (unlike in part b), where low-minute players had extreme RAPM values. The scatter plot also shows this improvement. There are fewer extreme RAPM values among players with limited playing time. In the end, ridge regression successfully reduces overfitting and produces more stable estimates of player impact.

Problem 2: Possession Weights

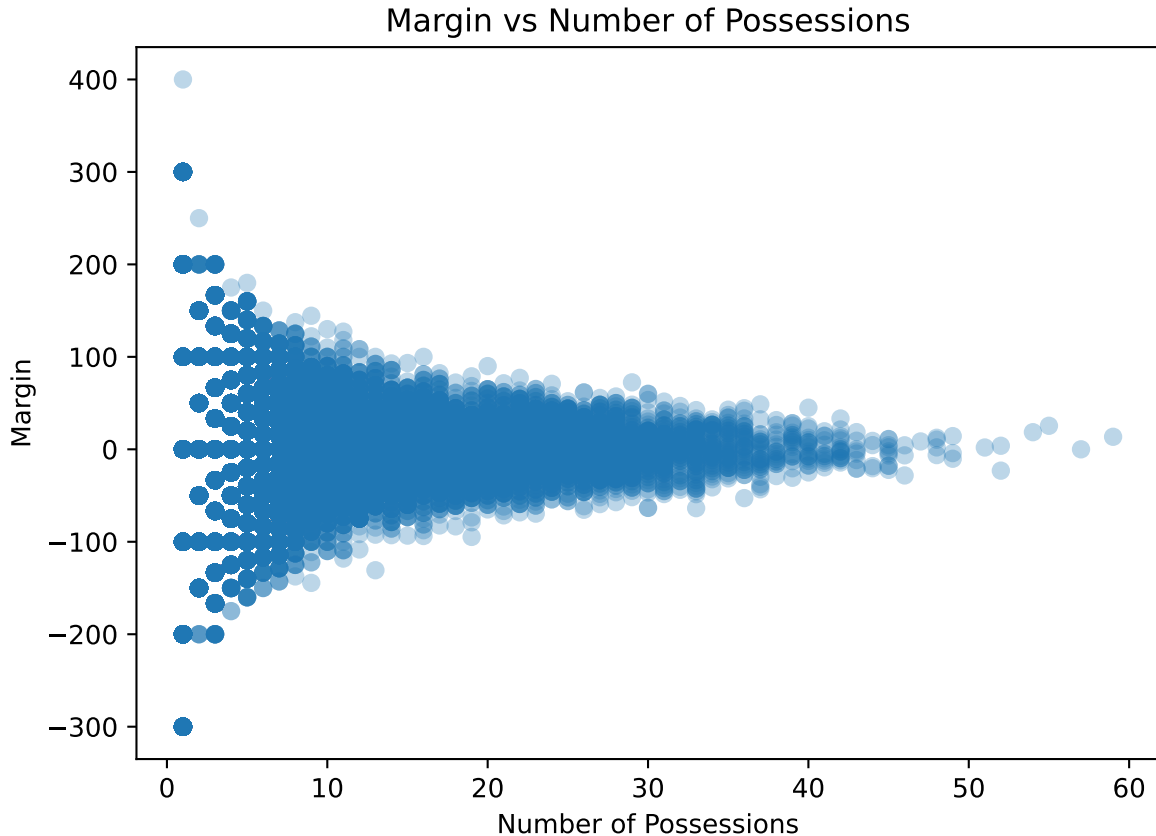
- (a) **Heteroscedasticity in Stints:** Make a plot of the 'margin' variable as a function of the number of possessions in a stint. What do you notice about the variance of 'margin'? Why do you think it is happening?

```
plt.figure(figsize=(7,5))

plt.scatter(nba_stints["n_pos"], nba_stints["margin"], alpha=0.3)

plt.xlabel("Number of Possessions")
plt.ylabel("Margin")
plt.title("Margin vs Number of Possessions")

plt.show()
```



The plot shows that for a small number of possessions the spread of the margin is large (high variance). The margin gradually narrows as the number of possessions increases. So, stints

with only a few possessions can have very extreme positive or negative margins, while longer stints tend to stay closer to zero. This happens because shorter stints are way more affected by randomness. With only a few possessions, a couple of made or missed shots can create a large swing in the margin. As the number of possessions increases, the effects of randomness wears off, making the margin more stable. The variability of the outcome changes with the number of possessions.

- (b) **Implementing Weights Regression:** The solution to the issue that you observed in 2(a) is something called **weighted least squares**. This involves adjusting the error term for each stint based on a weight that accounts for whatever factor controls the variance. The new weighted model looks like this:

$$\text{margin}_i \sim \text{Normal} \left(\sum_j c_{ij} (\text{RAPM})_j, \frac{\sigma^2}{w_i} \right)$$

where w_i is a coefficient that determines how the variance of ‘margin’ should scale for each stint. The idea is that w_i should be smaller for stints where the variance is high, and larger for stints where the variance is low. This forces the model to fit the low-variance stints more closely than the high-variance stints. The correct weights for this problem are the $w_i = \text{num_pos}_i$, which implies that the variance of the margin is inversely proportional to the number of stints. Verify this by making a scatter plot of the margin rate times the square root of the number of possessions versus the number of possessions. Then recalculate the player ‘RAPM’ coefficients from 1(c) by setting the ‘weights’ keyword in the model fit to be ‘n_pos’. How have the rankings of top players changed?

Checking if margin x sqrt(n_pos) is more stable than n_pos

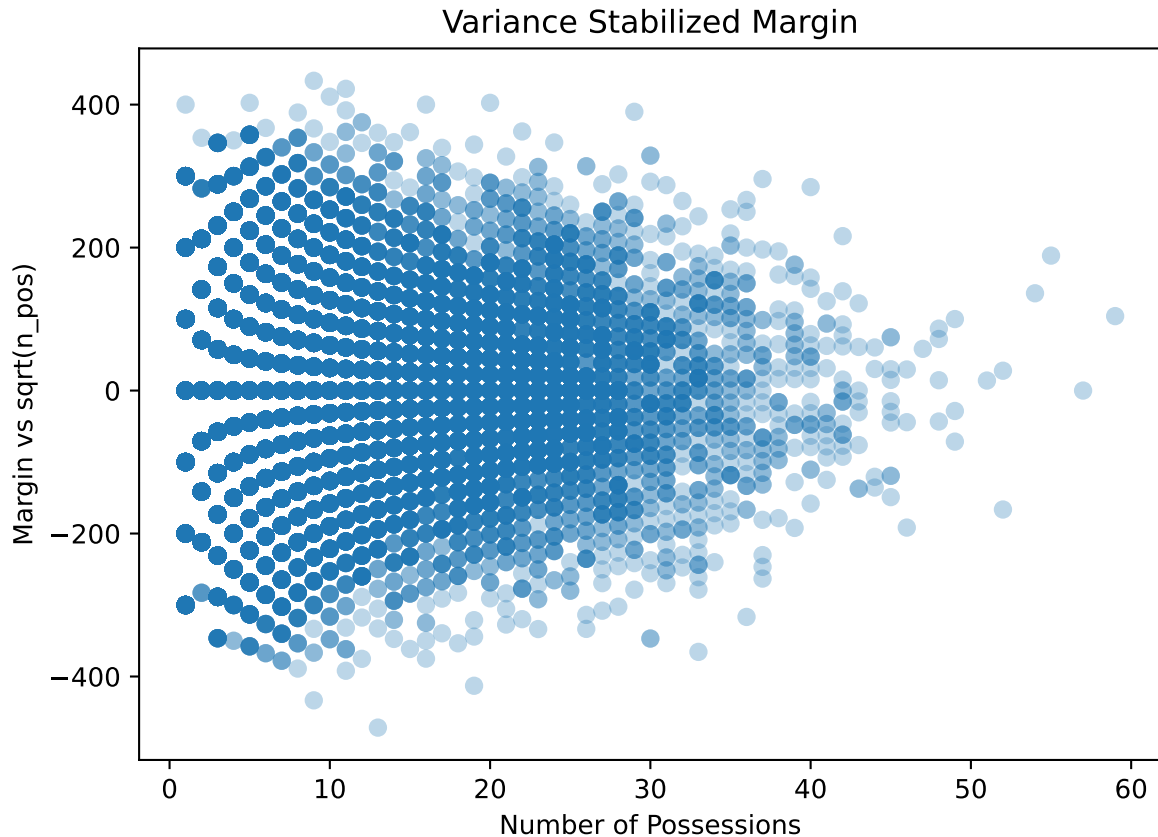
```
plt.figure(figsize=(7,5))

y_scaled = nba_stints["margin"] * np.sqrt(nba_stints["n_pos"])

plt.scatter(nba_stints["n_pos"], y_scaled, alpha=0.3)

plt.xlabel("Number of Possessions")
plt.ylabel("Margin vs sqrt(n_pos)")
plt.title("Variance Stabilized Margin")

plt.show()
```



After scaling the margin by the square root of the number of possessions, the spread of the data looks much more consistent across different n_pos . Now the variability no longer decreases with the number of possessions. There is still some variation, but the spread is much more uniform, telling us that the transformation is doing a good job stabilizing the variance.

Weighted Ridge Regression

```
ridge_w = RidgeCV(alphas=alphas, fit_intercept=True, cv=5)

ridge_w.fit(X, y, sample_weight=nba_stints["n_pos"])

print("Optimal alpha (weighted):", ridge_w.alpha_)
```

Optimal alpha (weighted): 2424.462017082331

New RAPM

```

rapm_w = pd.DataFrame({
    "player_id": X.columns,
    "RAPM": ridge_w.coef_
})

rapm_w = rapm_w.merge(players, on="player_id", how="left")

rapm_w["minutes"] = rapm["minutes"]

top20_w = rapm_w.sort_values(by="RAPM", ascending=False).head(20)

top20_w[["player_id", "player_name", "RAPM", "minutes"]]

```

	player_id	player_name	RAPM	minutes
258	203999	Nikola Jokic	4.581219	2255.54
2	203110	Draymond Green	4.500203	2019.37
156	203954	Joel Embiid	4.214207	2249.47
306	202695	Kawhi Leonard	3.933885	1817.22
167	203076	Anthony Davis	3.893304	1880.81
243	201950	Jrue Holiday	3.809015	2097.00
244	203507	Giannis Antetokounmpo	3.725310	1840.59
105	1631132	Christian Koloko	3.575357	757.82
419	1629627	Zion Williamson	3.436859	859.13
257	203932	Aaron Gordon	3.083740	1966.73
198	1628404	Josh Hart	3.050665	2474.72
242	201572	Brook Lopez	2.862314	2156.65
15	1628369	Jayson Tatum	2.671079	2771.44
26	1629661	Cameron Johnson	2.669018	1073.00
112	1630217	Desmond Bane	2.661300	1706.02
260	1629008	Michael Porter Jr.	2.577462	1686.43
16	1628401	Derrick White	2.571904	2078.65
301	201587	Nicolas Batum	2.562104	1814.79
344	1626246	Boban Marjanovic	2.541827	166.57
235	1630532	Franz Wagner	2.529214	2293.49

Scatter Plot

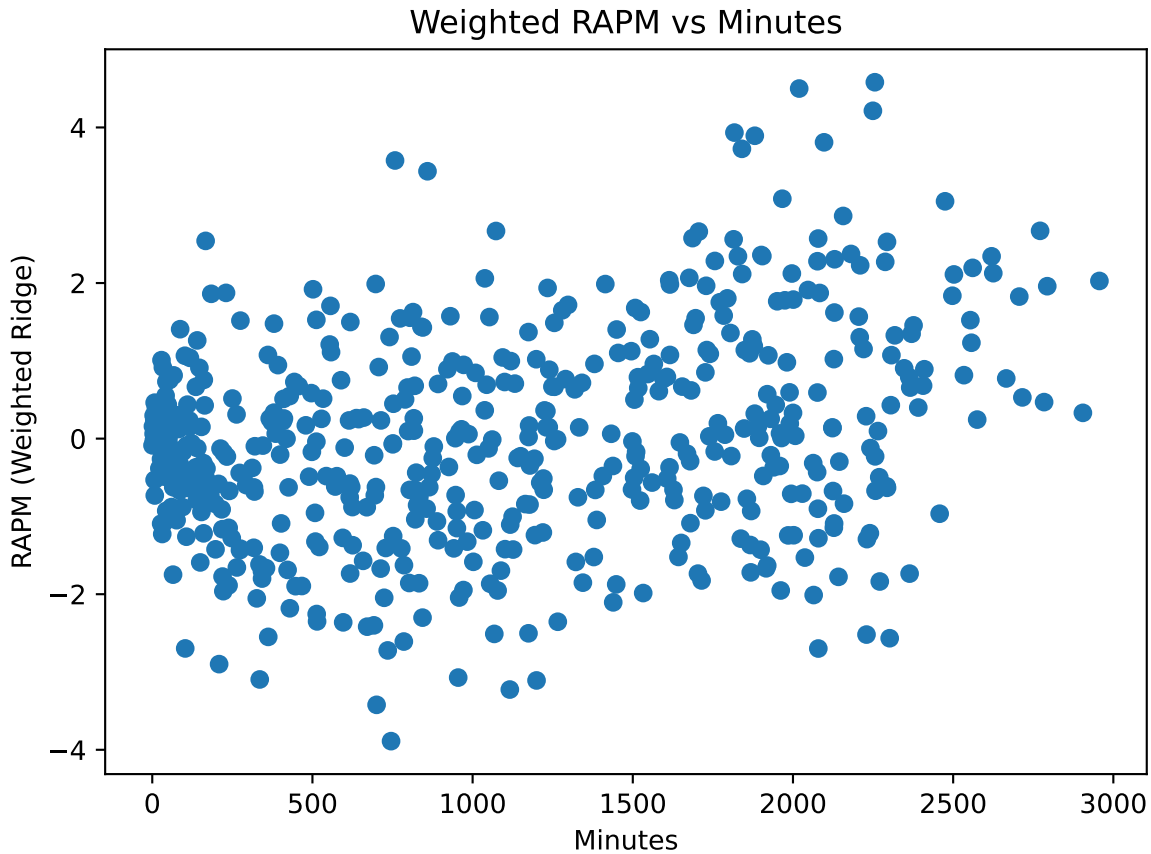
```

plt.figure(figsize=(7,5))
plt.scatter(rapm_w["minutes"], rapm_w["RAPM"])

plt.xlabel("Minutes")
plt.ylabel("RAPM (Weighted Ridge)")

```

```
plt.title("Weighted RAPM vs Minutes")  
plt.show()
```



The optimal value of alpha increased from around 701 to approximately 3665, showing that a stronger level of regularization was needed once the weights were introduced. With the weighted ridge regression, the top-ranked players are mostly well-established NBA stars such as Nikola Jokic, Joel Embiid, Giannis Antetokounmpo, Kawhi Leonard, and Anthony Davis, so the model is producing more realistic rankings. Compared to the previously used models, the rankings are now more heavily driven by players with higher minutes, which reflects the fact that longer stints provide more reliable information. There are still a few lower-minute players but they are way less dominant than before. By looking at the scatter plot, we can see the same improvement. Overall, combining weighting with ridge regression reduces the impact of noisy short stints and leads to more reliable estimates of player performance.

Problem 3: Interpreting Bootstrap Uncertainty

- (a) **Calculate Confidence Intervals Using the Bootstrap:** Suppose you are a general manager for a basketball team. You want to identify candidate players to add to your team, but you are unsure how certain to be about the ‘RAPM’ coefficients for the model. The bootstrap is a standard approach for calculating confidence intervals for model coefficients. Use either the ‘resample’ function from ‘sklearn’ or ‘np.random.choice’ along with a loop to calculate bootstrap samples of the model coefficients for each player. You may use the optimal ‘alpha’ found during the hyperparameter search in 2(b) for all bootstrap fits. Do not forget to resample the weights when bootstrapping!. For the top 20 players, display the RAPM estimates along with the confidence interval (calculate 92% intervals or some other high value that isn’t 95%). How much do the intervals of the top 20 overlap?

```
n_iterations = 100

coefs = []
```

Bootstrap Loop

```
for i in range(n_iterations):

    X_b, y_b, w_b = resample(
        X, y, nba_stints["n_pos"],
        replace=True,
        random_state=i
    )

    model_b = RidgeCV(alphas=[ridge_w.alpha_], fit_intercept=True)

    model_b.fit(X_b, y_b, sample_weight=w_b)

    coefs.append(model_b.coef_)
```

```
coefs = np.array(coefs)
```

Confidence Intervals

```
lower = np.percentile(coefs, 4, axis=0)
upper = np.percentile(coefs, 96, axis=0)
```


DataFrame

```
ci_df = pd.DataFrame({
    "player_id": X.columns,
    "RAPM": ridge_w.coef_,
    "lower": lower,
    "upper": upper
})

ci_df = ci_df.merge(players, on="player_id", how="left")

ci_df["minutes"] = rapm["minutes"]

ci_df = ci_df.rename(columns={"player_name_y": "player_name"})
```

Top 20 with CIs

```
top20_ci = ci_df.sort_values(by="RAPM", ascending=False).head(20)

top20_ci[["player_name", "RAPM", "lower", "upper", "minutes"]]
```

	player_name	RAPM	lower	upper	minutes
258	Nikola Jokic	4.581219	2.558932	6.556292	2255.54
2	Draymond Green	4.500203	2.744214	6.749457	2019.37
156	Joel Embiid	4.214207	2.423338	5.847386	2249.47
306	Kawhi Leonard	3.933885	2.084047	5.709762	1817.22
167	Anthony Davis	3.893304	2.367386	5.434205	1880.81
243	Jrue Holiday	3.809015	2.111632	5.652216	2097.00
244	Giannis Antetokounmpo	3.725310	1.551893	6.134397	1840.59
105	Christian Koloko	3.575357	1.119774	5.357411	757.82
419	Zion Williamson	3.436859	1.512390	5.264911	859.13
257	Aaron Gordon	3.083740	1.479029	4.702773	1966.73
198	Josh Hart	3.050665	1.167293	4.814753	2474.72
242	Brook Lopez	2.862314	0.339421	4.441491	2156.65
15	Jayson Tatum	2.671079	0.704471	4.820596	2771.44
26	Cameron Johnson	2.669018	1.014487	4.730741	1073.00
112	Desmond Bane	2.661300	0.653269	4.463688	1706.02
260	Michael Porter Jr.	2.577462	0.788732	4.294615	1686.43
16	Derrick White	2.571904	0.259672	4.366649	2078.65
301	Nicolas Batum	2.562104	0.813859	4.102944	1814.79
344	Boban Marjanovic	2.541827	1.062948	4.164013	166.57
235	Franz Wagner	2.529214	0.609111	4.328144	2293.49

The bootstrap confidence intervals for the top 20 players show quite some overlap. While players like Nikola Jokic, Draymond Green, and Joel Embiid consistently have high RAPM estimates, their intervals overlap with several other top players, such as Jrue Holiday, Kawhi Leonard, and Giannis Antetokounmpo, showing that although the model can identify a group of elite players, it is hard to be sure of how rank them within that group. The differences in RAPM values between many of these players are small relative to the uncertainty in their estimates. So, the results suggest that RAPM is useful for identifying top-tier players, but the exact order of the rank requires a careful interpretation.

- (b) **Impact of Minutes Played on Confidence Intervals:** One potential use of a model like this is to find players who do not play much but who might do well with more opportunity. Calculate the standard errors of each coefficient from the bootstrap samples and make a scatterplot of standard error versus minutes played. Comment on the relationship between minutes played and standard error- does it make sense to you intuitively and/or statistically?

```
se = np.std(coefs, axis=0)
```

```
ci_df["std_error"] = se
```

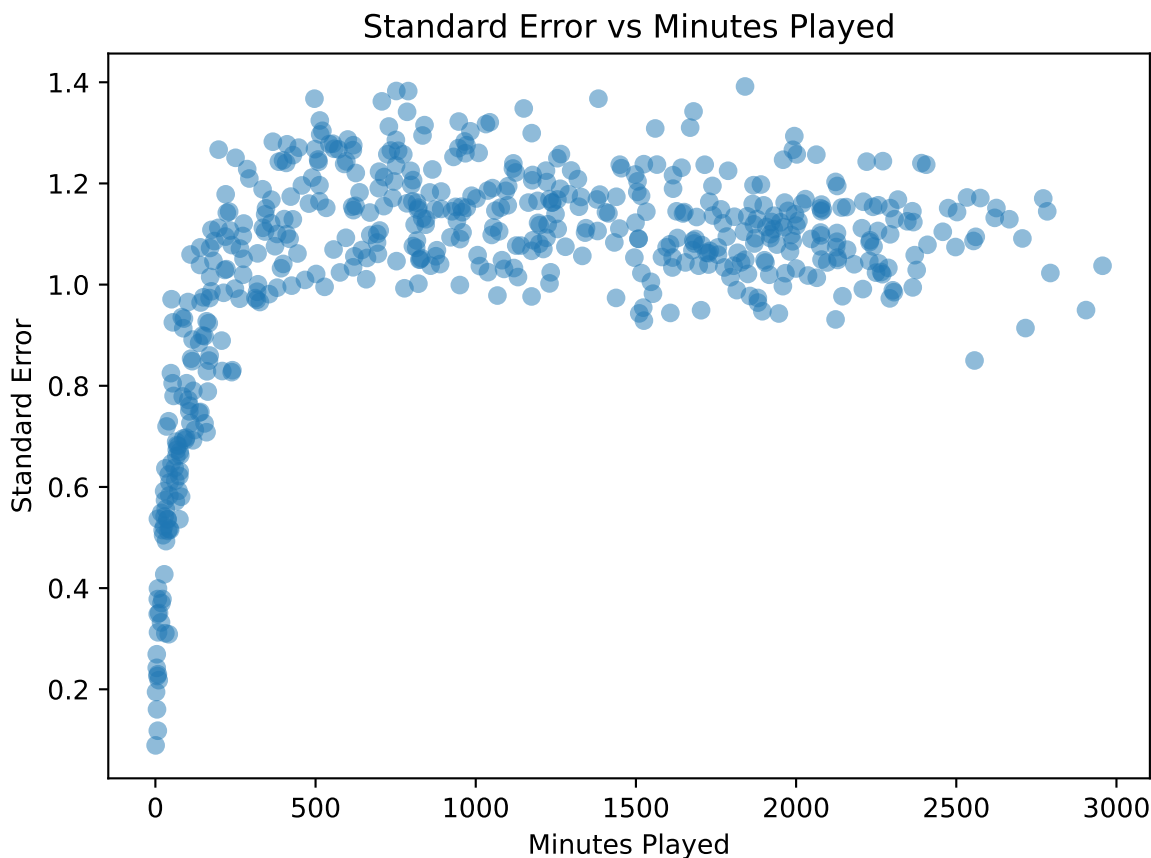
Scatter Plot

```
plt.figure(figsize=(7,5))

plt.scatter(ci_df["minutes"], ci_df["std_error"], alpha=0.5)

plt.xlabel("Minutes Played")
plt.ylabel("Standard Error")
plt.title("Standard Error vs Minutes Played")

plt.show()
```



The scatter plot shows that the relationship between minutes played and standard error has a pattern. Players with very low minutes played have the smallest standard errors, but as minutes played increase the standard error rises pretty fast and then stabilizes. Players with more than 500min have a relatively consistent standard error, with a slight decrease for the highest-minute players. This pattern makes sense statistically since players with very small number of minutes played appear in very few stints, which can sometimes lead to “fake” low variability in the bootstrap estimates. For players with more minutes, the model captures more variation in their performance, increasing the estimated uncertainty. Around the 500min mark, the estimates become more stable, and the standard errors level off or slightly decrease.

In the end, this tells us that estimates for players with very limited minutes can be misleading, and that more playing time generally leads to more reliable and stable RAPM estimates.

- (c) **Comparison to Bayesian Credible Intervals:** Bayesian statistics (which you should have learned about briefly in DATA 606) is a field of statistics based on an interpretation of probability as referring to uncertainty about the real world rather than as the frequency of outcomes in repeated experiments. In the Bayesian framework, it is normal to talk about the probability distribution of model parameters, which leads to the con-

cept of a ‘credible interval’, defined as an interval with a certain probability of containing the model parameter (a 92% credible interval would have a 92% chance of containing the model parameter). Credible intervals often correspond closely to confidence intervals captured using standard statistics, but this problem is one case where they diverge sharply. Ridge regression has an interpretation in Bayesian statistics, where the regularization parameter corresponds to the strength of a prior belief that the model coefficients have a normal distribution centered around zero and with variance inversely proportional to the regularization penalty α . For ridge regression interpreted as Bayesian statistics, there is an exact formula for the standard errors and confidence intervals which we can use to contrast with the bootstrap estimate. I have provided code below to calculate the standard errors and the Bayesian credible intervals (if you are curious about the full details [read this article](#)). You will need to adapt this code to your variable names and data structures:

```
# Here MarginVector is your Target
# StintMatrix are your observations
# WeightVector are your weights
# model_ridge is your selected model

# alpha_opt is the optimal regularization parameter from cross validation

weights = nba_stints["n_pos"].values
stint_mat = X.values
margin_vec = y.values

alpha_opt = ridge_w.alpha_

var = np.average(
    (margin_vec - ridge_w.predict(X))**2,
    weights=weights
)

num_players = stint_mat.shape[1]

AMat = (stint_mat * weights.reshape(-1, 1)).T @ stint_mat + alpha_opt * np.eye(num_players)

# This is the covariance matrix. You could find very high covariance between players on the s

posterior_covariance = var * np.linalg.inv(AMat)

se_bayes = np.sqrt(np.diag(posterior_covariance))
```

```

# add to dataframe
ci_df["se_bayes"] = se_bayes

# Plot comparison between Bayesian and Bootstrap
plt.figure(figsize=(7,5))

plt.scatter(ci_df["minutes"], ci_df["std_error"], alpha=0.5, label="Bootstrap")
plt.scatter(ci_df["minutes"], ci_df["se_bayes"], alpha=0.5, label="Bayesian")

plt.xlabel("Minutes Played")
plt.ylabel("Standard Error")
plt.title("Bootstrap vs Bayesian Standard Errors")

plt.legend()
plt.show()

```

Use the above code and calculate the Bayesian standard errors for each 'RAPM' coefficient. Make a plot of the Bayesian standard errors versus minutes played and compare to the standard errors calculated from the bootstrap. In which part of the range is the agreement highest? In which part is the disagreement highest? Focusing on the areas where the disagreement is highest, put yourself in the shoes of someone using the model and discuss which estimate of uncertainty is more realistic and why.

Answer (c) The comparison shows that the bootstrap and Bayesian standard errors have pretty different behaviors. The two methods have a similar behavior for players with moderate to high minutes, where both approaches produce relatively stable estimates, although the bootstrap values are consistently higher. However, for players with very low minutes there is quite a discrepancy. In this region, the bootstrap standard errors are much more variable and usually much smaller or larger, while the Bayesian standard errors follow a more smooth and stable pattern. This tells us that the bootstrap is more sensitive to noise and limited data in low-minute players. Practically speaking, the Bayesian standard errors seem more realistic, especially for players with few minutes. The ridge-based Bayesian approach includes a prior that shrinks estimates toward zero, which helps stabilize uncertainty when there is not much data. The bootstrap, however, relies entirely on the observed data, which can be sparse and noisy, leading to less reliable estimates when data is limited.

Extra Credit (5 Points): Ridge versus Lasso and Confidence/Credible Interval Calculations

- (a) An alternative to ridge regression that is used for variable selection is called the Lasso. The Lasso differs penalty causes a large number of model coefficients to be zero, making it

a good tool for variable selection and for creating interpretable models. Use ‘LassoCV’ to calculate the ‘RAPM’ coefficients. I recommend using regularization weights that range between 10^{-4} and 10^2 (the scale needs to be different compared to ridge regression). Compare the ‘RAPM’ values calculated with lasso to those calculated with ridge regression. Are there any notable differences in the top 20 players? Find the 10 players with the largest difference between lasso RAPM and Ridge RAPM in both directions (ridge greater than lasso and lasso greater than ridge). Within the players where there was large disagreement, consider how the players performed in the next NBA season (you may do this however you like, I recommend reading media reports about those players during the next season) and determine which model was more correct when it comes to players with large disagreements.

Fit Lasso

```
alphas_lasso = np.logspace(-4, 2, 50)

lasso = LassoCV(alphas=alphas_lasso, cv=5, max_iter=5000)

lasso.fit(X, y)

print("Optimal alpha (lasso):", lasso.alpha_)
```

Optimal alpha (lasso): 0.04941713361323833

```
rapm_lasso = pd.DataFrame({
    "player_id": X.columns,
    "RAPM_lasso": lasso.coef_
})

rapm_compare = rapm_lasso.merge(
    rapm_w[["player_id", "RAPM"]],
    on="player_id",
    how="left"
)

rapm_compare = rapm_compare.merge(players, on="player_id", how="left")

rapm_compare["diff"] = rapm_compare["RAPM"] - rapm_compare["RAPM_lasso"]
```

Ridge > Lasso

```
top_ridge_gt = rapm_compare.sort_values(by="diff", ascending=False).head(10)
top_ridge_gt[["player_name", "RAPM", "RAPM_lasso", "diff"]]
```

	player_name	RAPM	RAPM_lasso	diff
262	Christian Braun	-0.847571	-5.591374	4.743803
96	Davion Mitchell	-1.985147	-6.077339	4.092192
142	Devin Vassell	-3.225544	-7.289316	4.063772
332	Aaron Wiggins	-0.657098	-4.424444	3.767346
344	Boban Marjanovic	2.541827	0.000000	2.541827
23	Devin Booker	2.346412	0.000000	2.346412
314	Jarrett Allen	2.280281	0.000000	2.280281
249	Jordan Nwora	-2.502680	-4.739745	2.237065
45	Clint Capela	1.547707	-0.675133	2.222839
164	LeBron James	2.343837	0.190921	2.152917

Lasso > Ridge

```
top_lasso_gt = rapm_compare.sort_values(by="diff").head(10)
top_lasso_gt[["player_name", "RAPM", "RAPM_lasso", "diff"]]
```

	player_name	RAPM	RAPM_lasso	diff
156	Joel Embiid	4.214207	10.617025	-6.402818
99	Pascal Siakam	1.836803	7.637670	-5.800867
318	Kevin Love	0.707117	6.223765	-5.516648
48	Trae Young	1.825857	7.088164	-5.262307
258	Nikola Jokic	4.581219	9.094526	-4.513307
218	Jalen Brunson	0.681002	4.757550	-4.076547
22	Chris Paul	1.138326	4.812508	-3.674182
406	Darius Garland	2.227693	5.632242	-3.404549
339	Isaiah Joe	1.277819	4.601979	-3.324161
168	Lonnie Walker IV	-3.108680	0.000000	-3.108680

The optimal lasso alpha is small (around 0.049), but the model still produces much more extreme coefficients compared to ridge. The main differences show that lasso tends to push some players to very large positive or negative values, while ridge keeps more moderate estimates. For example, Christian Braun, Devin Vassell, and Davion Mitchell have much more negative RAPM under lasso, while ridge assigns them smaller negative values. On the other hand, star players such as Joel Embiid, Nikola Jokic, Trae Young, and Pascal Siakam receive much larger positive RAPM values under lasso than under ridge. This means that lasso is more aggressive and tends to amplify strong signals while decreasing others toward zero. Ridge, on the other hand, produces more stable and balanced estimates across players.

Looking at players with large disagreements, ridge seems to be more realistic in many cases. For example, Christian Braun and Davion Mitchell did not perform at extremely poor levels in the following season, meaning that the very negative lasso estimates may be too extreme. On the other hand, for top players such as Joel Embiid and Nikola Jokic, lasso assigns much higher RAPM values, which actually reflects their elite impact (with some values still exaggerated).

In the end, I think ridge is more reliable for general player evaluation since it avoids extreme estimates, while lasso can sometimes overreact to strong signals, especially in smaller or noisier samples.

- (b) There is a subtle conceptual flaw in how the confidence or credible intervals were calculated in this lab. This is apparent if you paid very close attention during the meetup. Can you tell me what it is?

The conceptual flaw is that the estimates were calculated after selecting the model and tuning alpha. This means that the intervals do not consider the uncertainty introduced by the model selection. So, we are treating the chosen model as if it were fixed, when in fact it was selected based on the data. This leads to very optimistic or very small intervals, because they ignore the variability that comes from choosing the model and parameters.